Retrospective

What Went Well

- **Architectural Pivot:** Successfully transitioned the frontend from Streamlit to React (Vite) to support high-performance Knowledge Graph visualizations.
- **Core Backend & Data:** Established a robust FastAPI structure with functional routes for portfolio management and risk analytics.
- **Causal Reasoning:** Integrated Neo4j for graph-based impact tracing, allowing the system to map "Butterfly Events" across global industries.
- **Data Enrichment:** Leveraged Python logic to incorporate geospatial broadband metrics, adding a unique infrastructure-risk dimension to scoring.
- **Team Synergy & Efficient Execution:** Maintained strong collaboration and effective pair programming despite a remote environment, leading to increased team bonding.

Retrospective

What Could Be Improved

- **Coding Redundancies & Sync:** Identified overlapping logic between FastAPI routes and AI agents. A centralized utility structure and unified schema are needed to eliminate redundancy and resolve Pydantic-to-React mapping delays.
- **Mandatory Meeting Alignment:** Presentation deliverables, sprint goals, and detailed “work-done” updates should be mandatory agenda items in every meeting to ensure team-wide alignment.
- **Task Granularity:** Complex technical tasks, such as LLM Reasoning and Causal Modeling, require finer sub-task breakdowns for more accurate effort estimation and tracking.
- **Communication Flow:** Standardizing status update frequency during meetings will help identify and resolve technical blockers early, preventing delays in the final demo.

Retrospective

Action Items for Next Sprint

- **Process Rigor:** Institute mandatory weekly technical summaries and mid-sprint "blocker" check-ins to ensure deliverable alignment.
- **Planning & Availability:** Establish better meeting windows and ensure all stories have clear Acceptance Criteria before development begins.
- **Velocity Adjustment:** Plan for fewer story points to improve team velocity and allow for iterative presentation updates.
- **Performance & Docs:** Allocate dedicated time for modular code refactoring and optimization alongside "Documentation Power Hours" to finalize the Technical Paper and GitHub Wiki.